

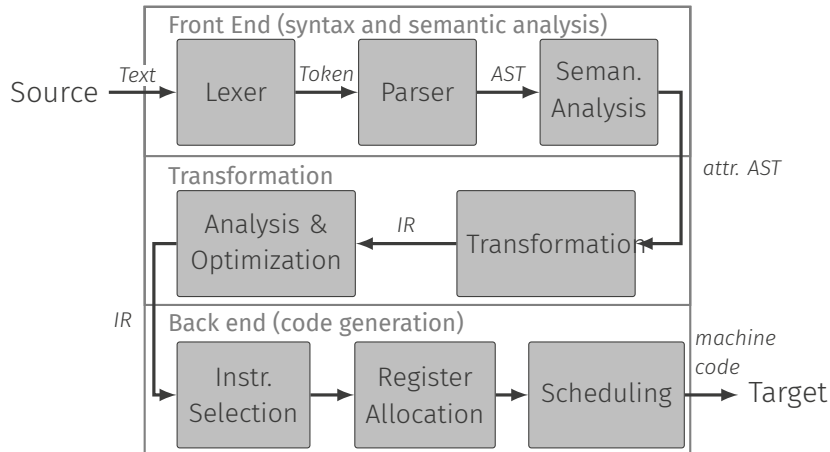
Lexical Analysis

Roland Leißa

(based on slides by S. Hack, R. Wilhelm, M. Sagiv, G. Snelting)

University of Göttingen

Overview



Foundations

Implementation

Generators

Implementation Details

Practical Problems

Links

Lexical Analysis (Scanning)

- Functionality

Input: program as sequence of characters

Output: program as sequence of symbols (tokens)

- Report errors, symbols illegal in the programming language
- Additional bookkeeping:
 - Identify language keywords and standard identifiers
 - Eliminate “whitespace”, e.g., consecutive blanks and newlines
 - Track text coordinates for error report generation
 - Construct table of all symbols occurring (symbol table)

Foundations

Notations

A **language** L is a set of **words** x over an **alphabet** Σ .

$a_1a_2 \dots a_n$ a **word** over Σ , $a_i \in \Sigma$

ε the empty word

Σ^n the words of length n over Σ

Σ^* the set of finite words over Σ

Σ^+ the set of non-empty finite words over Σ

$x.y$ the concatenation of words x and y

Language Operations

$L_1 \cup L_2$	Union
$L_1 L_2 = \{x.y \mid x \in L_1, y \in L_2\}$	Concatenation
$\bar{L} = \Sigma^* \setminus L$	Complement

$$\begin{aligned} L^0 &= \{\varepsilon\} \\ L^n &= \{x_1 \dots x_n \mid x_i \in L, 1 \leq i \leq n\} \\ L^* &= \bigcup_{n \geq 0} L^n \\ L^+ &= L^* \setminus L^0 \end{aligned} \quad \text{Closure}$$

Regular Languages

Defined inductively

- $\emptyset$ is a regular language over Σ
- $\{\epsilon\}$ is a regular language over Σ
- For all $a \in \Sigma$, $\{a\}$ is a regular language over Σ
- If R_1 and R_2 are regular languages over Σ , then so are:
 - $R_1 \cup R_2$
 - $R_1 R_2$
 - R_1^*

Regular Expressions and the Denoted Regular Languages

Defined inductively

- $\emptyset$ is a regular expression over Σ denoting $\emptyset$,
- ϵ is a regular expression over Σ denoting $\{\epsilon\}$,
- For all $a \in \Sigma$, a is a regular expression over Σ denoting $\{a\}$,
- If r_1 and r_2 are regular expressions over Σ denoting R_1 and R_2 , resp., then so are:
 - $(r_1 | r_2)$, which denotes $R_1 \cup R_2$,
 - $(r_1 r_2)$, which denotes $R_1 R_2$, and
 - $(r_1)^*$, which denotes R_1^* .

Example

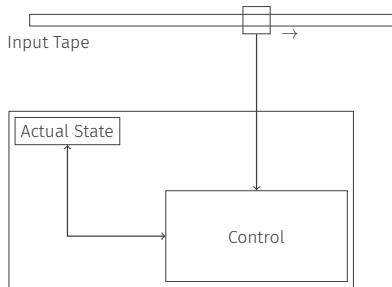
Expression	Language	Example words
$a b$	$\{a, b\}$	a, b
ab^*a	$\{a\}\{b\}^*\{a\}$	$aa, aba, abba, abbbba, \dots$
$(ab)^*$	$\{ab\}^*$	$\epsilon, ab, abab, \dots$
$abba$	$\{abba\}$	$abba$

Regular Expressions: Common Extensions

- R^+ Like R^* but does **not** match zero-sized strings.
- $^$ Matches the starting position within the string.
- $.$ Matches any single character (usually including newlines).
- $[]$ Matches a single character that is contained within the brackets. For example, $[abc]$ matches **a**, **b**, or **c**. $[a-z]$ specifies a range which matches any lowercase letter from **a** to **z**.
- $[^]$ Matches a single character that is not contained within the brackets. For example, $[^abc]$ matches any character other than **a**, **b**, or **c**.
- $\$$ Matches the ending position of the string or the position just before a string-ending newline.

- process **input**
- make **transitions** from configurations to configurations;
- **configurations** consist of (the rest of) the input and some **memory**;
- the **memory** may be small, just one variable with finitely many values,
- but the memory may also be able to grow without bound, adding and removing values at one of its ends;
- the type of memory determines its ability to **recognize** a class of languages

Finite State Machine



- simplest type of automaton
- memory consists of only one variable
 - stores one out of finitely many **states**

Non-Deterministic Finite-State Machine (NFSM)

$M = (\Sigma, Q, \rightarrow, q_0, F)$ where:

- finite **alphabet**: Σ
- finite set of **states**: Q
- **initial state**: $q_0 \in Q$
- **final states**: $F \subseteq Q$
- **transition relation**: $q \xrightarrow{a} p$ where $q, p \in Q$ and $a \in \Sigma \cup \{\epsilon\}$

Transition Diagram

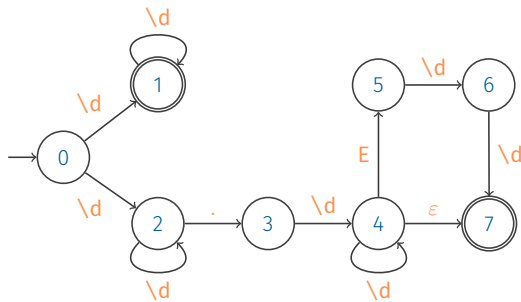
- Chain all transitions nicely together
- q_0 has a special “entry” mark
- final states are doubly encircled

Example: Integer and Real Constants

	$\backslash d$	.	E	ϵ
0	{1,2}	$\emptyset$	$\emptyset$	$\emptyset$
1	{1}	$\emptyset$	$\emptyset$	$\emptyset$
2	{2}	{3}	$\emptyset$	$\emptyset$
3	{4}	$\emptyset$	$\emptyset$	$\emptyset$
4	{4}	$\emptyset$	{5}	{7}
5	{6}	$\emptyset$	$\emptyset$	$\emptyset$
6	{7}	$\emptyset$	$\emptyset$	$\emptyset$
7	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

$$q_0 = 0$$

$$F = \{1, 7\}$$



Finite-State Machines vs Scanners

Finite-state machines

- get an input word
- start in their initial state
- make a series of transitions under the characters constituting the input word
- accept (or reject)

Scanners

- get an input string
- start in their initial state
- attempt to find the end of the next word
- when found, restart in their initial state with the rest of the input
- terminate when the end of the input is reached or an error is encountered

Maximal Munch Strategy

Find longest prefix of remaining input that is a legal symbol

- first input character of the scanner:
first “non-consumed” character
- in final state, and exists transition under the next character:
make transition and remember position
- in final state, and no transition under the next character:
symbol found
- actual state not final and no transition under the next character: backtrack to last passed final state
 - There is none: Illegal string
 - Otherwise: Actual symbol ended there

Warning: Certain overlapping symbol definitions will result in quadratic runtime:
→ exercise

The Language Accepted by a Finite-State Machine

- $M = (\Sigma, Q, \rightarrow, q_0, F)$
- For $q \in Q, w \in \Sigma^*$, (q, w) is a **configuration**
- The binary relation $\vdash_M$ on configurations is defined by:

$$\text{STEP} \frac{q \xrightarrow{a} p}{(q, aw) \vdash_M (p, w)}$$

- The **language accepted** by M

$$L(M) = \{w \in \Sigma^* \mid \exists q_f \in F : (q_0, w) \vdash_M^* (q_f, \epsilon)\}$$

Theorem

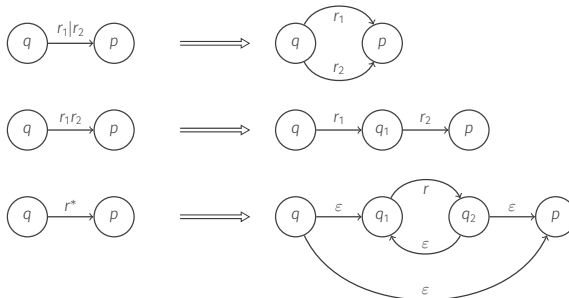
For every regular expression r , there exists an NFSM that accepts the regular language defined by r .

A Constructive Proof (Algorithm)

- A regular language is defined by a regular expression r
- Construct an “NFSM” with one final state, q_f , and the transition



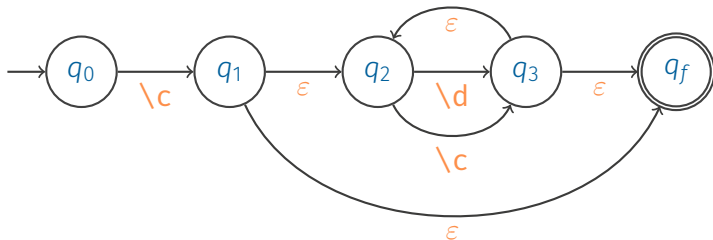
- Decompose r and develop the NFSM according to the following rules



until only transitions under single characters and ϵ remain.

Examples

Identifier: $\backslash c(\backslash c|\backslash d)^*$ where $\backslash c = \underbrace{(a|\dots|z|A|\dots|Z)}_{[a-zA-Z]}$ and $\backslash d = \underbrace{(0|\dots|9)}_{[0-9]}$



Deterministic Finite-State Machine (DFSM)

- No ϵ -transitions
- At most one transition from every state under a given character
i.e.: $\Rightarrow$ is a deterministic version of $\rightarrow$

Theorem

For every NFSM, $M = (\Sigma, Q, \rightarrow, q_0, F)$ there exists a DFSM, $M' = (\Sigma, Q', \Rightarrow, q'_0, F')$ such that $L(M) = L(M')$.

Constructive Proof (Subset Construction):

Construct a DFSM whose states are **sets of states** of the NFSM.

The DFSM simulates all possible transition paths under an input word in parallel.

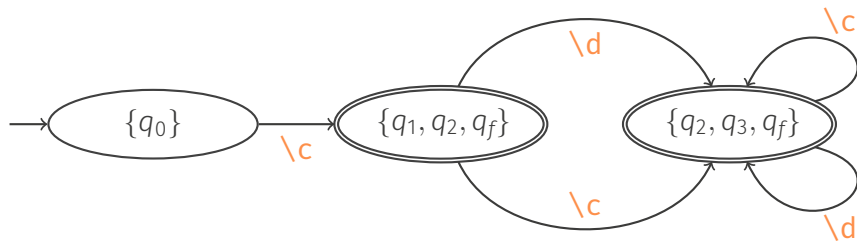
The Construction Algorithm

- Starts with $q'_0 = \epsilon\text{-Closure}(q_0)$ as the initial DFSM state where $\epsilon\text{-Closure}(q) = \{p \mid (q, \epsilon) \vdash_M^* (p, \epsilon)\}$ is the set of ϵ -successors
- Iteratively create more states and more transitions
- For each DFSM state $S \subseteq Q'$ already constructed and character $a \in \Sigma$:

$$\delta \frac{S' = \epsilon\text{-Closure}(\{p \mid q \xrightarrow{a} p, \text{ where } q \in S\})}{S \xRightarrow{a} S'}$$

- Only add new state if not previously constructed
- A DFSM state S is accepting (in F') if there exists $q \in S$ such that $q \in F$
- Complexity: $\mathcal{O}(2^{|Q|})$ (for lexer generation linear in practice)

Example: $\backslash c(\backslash c|\backslash d)^*$, NFSM $\rightarrow$ DFSM



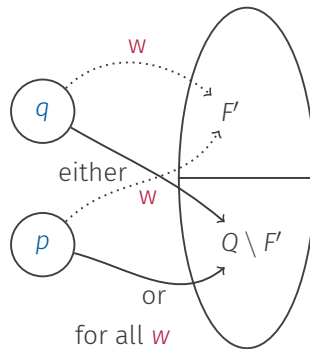
DFSM Minimization

DFSM need not have minimal size, i.e. minimal number of states and transitions

- q and p are **indistinguishable** (have the same acceptance behavior) iff

for all words w : $(q, w) \vdash_M^*$ and $(p, w) \vdash_M^*$ lead into either F' (**accepting**) or $Q' \setminus F'$ (**rejecting**).

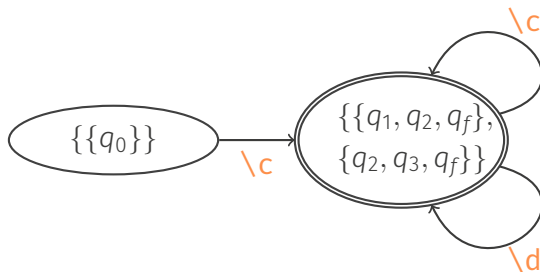
- Indistinguishability is an equivalence relation
- **Goal**: merge undistinguishable states $\equiv$ consider equivalence classes as new states



DFSM Minimization Algorithm

- Input a DFSM $M = (\Sigma, Q, \Rightarrow, q_0, F)$
- Iteratively refine a **partition** of the set of states, where each set in the partition consists of states **so far undistinguishable**
- Start with the partition $\Pi = \{F, Q \setminus F\}$
- Refine the current Π by splitting sets $S \in \Pi$ if there exist $q_1, q_2 \in S$ and $a \in \Sigma$ such that
 - $\{\dots, q_1, \dots\} \xRightarrow{a} S_1$ and $\{\dots, q_2, \dots\} \xRightarrow{a} S_2$ and $S_1 \neq S_2$
- Merge sets of undistinguishable states into a single state
- Complexity: quadratic. There exist algorithms with $\mathcal{O}(|\Sigma||Q| \log |Q|)$.

Example: $\backslash c(\backslash c|\backslash d)^*$, DFSA Minimization



Implementation

Table Implementation

- Goal: Make transitions in $\mathcal{O}(1)$
- Representation as adjacency matrix:
 - Size = $|S| * |\Sigma|$
 - With one byte per character: $|\Sigma| = 256$

Example FSM:

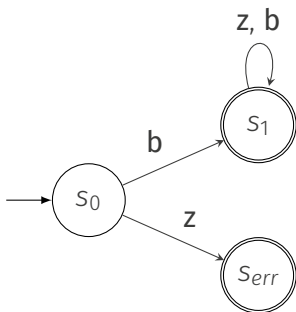


Table:

State	Input char		
	b	z	Misc
0	1	Err	Err
1	1	1	Err

Table Implementation

```
int table[|S|][|Σ|];
char c = next_char();
int result = ERROR;
int state = 0;
while (true) {
    if (is_final(state))
        result = state;

    state = table[state][c];
    if (is_error(state))
        break;
    c = next_char();
}
return result;
```

Table:

State	Input char		
	b	z	Misc
0	1	Err	Err
1	1	1	Err

- “Maximum munch”:
 - Iterate until error
 - Return last valid final state
- **Missing here:** Seen final state but cannot continue and not in final state → backtrack (maximal munch)

Implementation as Program

```
char c;
int result = ERROR;
s0:
switch (c) {
    case 'b': c = next_char(); goto s1;
    case 'z': goto end;
    default: goto end;
}
s1:
result = 1;
switch (c) {
    case 'b': c = next_char(); goto s1;
    case 'z': c = next_char(); goto s1;
    default: goto end;
}
end:
return result;
```

Table:

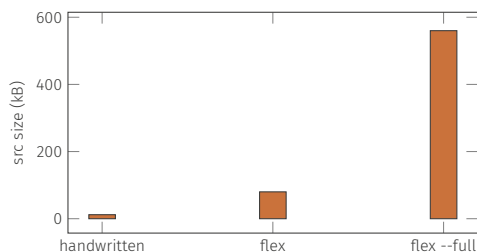
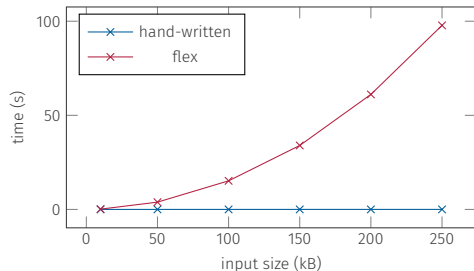
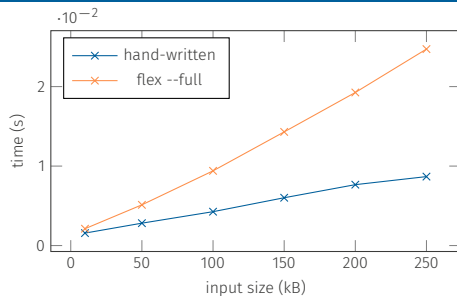
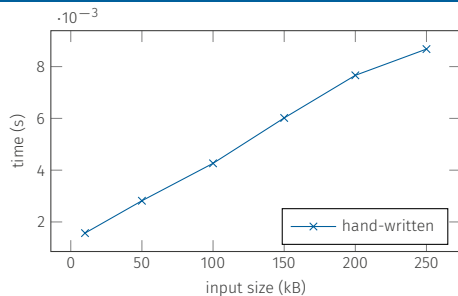
State	Input char		
	b	z	Misc
0	1	Err	Err
1	1	1	Err

- States turn into jump labels $\Rightarrow$ State implicit in program point
- rows of table as **switch**
(Compiler implements jump table)

Direct Construction of Lexer Code from Regexps

- `a`
`if accept("a"):`
 `# ...`
- `ab`
`if accept("a"):`
 `if accept("b"):`
 `# ...`
- `aB`
`if accept("a"):`
 `# match B`
- `(aA)*`
`while accept("a"):`
 `# match A`
- `aA|bB`
`if accept("a"):`
 `# match A`
`if accept("b"):`
 `# match B`
- $(axA)|(ayB) = a((xA)|(yB))$
`if accept("a"):`
 `if accept("x"):`
 `# match A`
 `if accept("y"):`
 `# match B`

Performance: Hand-Written vs Generated Lexer (Project)



Generator

- Allows for compact and declarative specification of lexing:
 - Specify each token as a regular expression
 - Perform **action** when token is recognized (specified by code)
- Automatically computes deterministic minimal FSM

Examples

- **Lex**: One of the first
- **Flex**: Extension of **Lex**, actively developed
- Many more for different languages

Example Lexer For XML

Example:

```
<?xml version="1.0"?>  
<!-- my personal books -->  
<books>  
    <book name='Goedel, Escher, Bach' />  
</books>
```

Example Lexer For XML

Example:

```
<?xml version="1.0"?>  
<!-- my personal books -->  
<books>  
  <book name='Goedel, Escher, Bach' />  
</books>
```

 : Strings

 : XML Names

 : Other token kinds

Flex Specification and Usage

```
/* Define regular expressions */
NameStartChar  [a-zA-Z]
NameChar       {NameStartChar}|[\\-\\.0-9]
Name           {NameStartChar}({NameChar})*
Comment        "<!--"([^\-]|"-"[^\-])*"-->"
String         \'[^\']*\'|\"[^\"]*\"
Token          "<?"| "</"| "<"| ">/"| ">"| "?>"| "="

%%
/* Actions on matched expressions */
{String}       printf("String: %s\\n", yytext);
{Name}         printf("Name: %s\\n", yytext);
{Comment}      /* skip */
{Token}        printf("Token: %s\\n", yytext);
.|\\n          /* skip */

%%
/* Main program */
int main(void) {
    yylex();
    return 0;
}
```

```
$ flex -o scanner.c scanner.l
$ cc scanner.c -lfl -o scanner
$ ./scanner < test.xml
Token: <?
Name: xml
Name: version
Token: =
String: "1.0"
Token: ?>
Token: <
Name: books
Token: >
Token: <
Name: book
Name: name
Token: =
String: 'Goedel, Escher, Bach'
Token: />
Token: </
Name: books
Token: >
```

Implementation Details

Token = (Tag, Value, Location)

- Tag:
 - The terminal symbol for the parser
 - Represented by an **enum** value
- Value:
 - Constants: String or numerical value
 - Caveat:** Target machine might have different numerical range
 - Identifier: String or better pointer to string table
 - Else: Not needed (**nullptr**)
 - Use **union** to save space
- Location:
 - Not needed for compilation but error reporting
 - Contains source file, line number, column number

Input Stream and Token Stream

Input character stream:

- Character-wise: Potentially slow because of function call to library for each character
- Buffered: Read in larger blocks. Has larger bookkeeping
- Memory mapped: OS maps input file into virtual memory
Fast, easy, but potentially less portable
- SIMD to skip comments quickly

Output token stream:

- Mostly in pull mode: Parser calls lexer for next token
(Is ok performance-wise, lexer has little state so not many cache misses)

The String Table

Maps identical identifiers to same entry (string interning).

Goals:

- Save space because identifiers typically appear more than once
- Replace costly string comparisons by pointer comparisons
- Store additional information about identifiers
 - Is identifier keyword? ($\Rightarrow$ treating keywords as identifiers simplifies lexer)
 - Current definition during semantic analysis

Implementation:

- Use hash table for $\mathcal{O}(1)$ access

Practical Problems

Nested Comments

```
/* Nested comments /* in Rust */ are cool!*/
```

- Not a regular language
- Tools like flex: state variables
- Hand-written lexer: easy, just add a counter

When Whitespace Carries Meaning

- Some languages use indentation to delimit blocks (instead of braces)
- Examples: Python, Haskell, Ada

Python example:

```
if not name:
    name = "a.txt"
    with open(name) as f:
        for line in f:
            print line
            count += 1
```

Tokens:

```
<if> <not> <id, name> <:> <newline>
<indent> <id, name> <=> <string, a.txt> <newline>
<with> <id, open> <(> <id, name> <)> <as> <id, f> <:> <newline>
<indent> <for> <id, line> <in> <id, f> <:> <newline>
<indent> <print> <id, line> <newline>
<id, count> <+=> <number, 1> <newline>
<dedent> <dedent> <dedent>
```

Possible implementation:

- Lexer remembers current indent level (context-sensitive)
- When depth changes, emit indent or dedent token(s)
- Parser stays context-free

Undesirable Lexer/Parser Dependencies

Java/C++: Types `List<List<Integer>>` and `vector<vector<int>>`:

- Maximum munch yields `>>` as a token
- Parser actually expects two tokens `> >`
- Solution: Parser decomposes `>>` on demand

C/C++: Expression `(A)*B` ambiguous without type information:

- Type cast of dereferenced pointer: `(int)*ptr`
- Multiplication with operand in parens: `(x)*y`

Possible solution:

- Lexer uses uniform token for types and identifiers
- Parser accesses symbol table to figure out if `A` is a type or not (have fun with `typedef`).
- Causes complications when using parser generators

Links

- <https://cyberzhg.github.io/toolbox/>